

Segurança da Informação & Consultoria Web3

Relatório de Auditoria de Segurança

ERC1155ReceivableToken

Cliente	AxiaDigitalSolutions
Preparado por	HeimdallSec
Data	23/07/2026
Versão do Documento	1.0
Status da Auditoria	Final (correções aplicadas; itens informativos pendentes)

Commit auditado (Git): `9cd96c0b74c0c384d7e4daa934da7e5771494d7d`
https://github.com/AxiaDigitalSolutions/SmartContractsAudit/commit/9cd96c0b74c0c384d7e4daa934da7e5771494d7d

DOCUMENTO CONFIDENCIAL

Sumário

Sumário 2
1. Resumo Executivo 3
1.1 Visão Geral do Escopo 3
1.2 Resumo das Descobertas 3
2. Metodologia de Auditoria 3
2.1 Análise Geral do Contrato 4
3. Descobertas Detalhadas (Findings) 4
[HSEC-ERT-01] Ausência de `whenNotPaused` em Transferências — Pausa Não Bloqueia Transferências ERC-1155 5
[HSEC-ERT-02] MintBatch Sem Limite de Tamanho de Array — Possível Block Gas Limit DoS 5
[HSEC-ERT-03] Risco de Centralização — Admin Único com Controle Total Sobre TokenIds e URIs 6
[HSEC-ERT-04] Risco de Centralização — Admin Único com Todos os Papéis (Single Point of Failure) 7
[HSEC-ERT-05] URI Configurável Após Mint — Risco de Integridade de Metadados 8
[HSEC-ERT-06] Mint de TokenId Sem URI Definida — Token "Fantasma" Sem Metadados 9
[HSEC-ERT-07] Ausência de Função `renounceRole` — Possibilidade de Auto-Revogação Irreversível 10
[HSEC-ERT-08] Versão do Solidity com Pragmatismo Amplo — ^0.8.0 Permite Versões Instáveis 11
4. Recomendações Gerais e Melhores Práticas 11
5. Disclaimer — Análise de Ameaças por Ação 11
6. Termo de Isenção de Responsabilidade (Disclaimer) 12

1. Resumo Executivo

A HeimdallSec foi contratada pela **AxiaDigitalSolutions** para conduzir uma auditoria de segurança no contrato **ERC1155ReceivableToken**. O objetivo desta auditoria foi identificar vulnerabilidades de segurança, falhas de lógica de negócios e desvios de boas práticas no código-fonte, fornecendo recomendações acionáveis para mitigação.

Este relatório detalha as descobertas feitas durante o período de 23/07/2026. A auditoria combinou análise estática automatizada e revisão manual rigorosa por especialistas em segurança Web3.

O contrato implementa um token ERC-1155 para ativos de recebíveis/charges, com tokenId único por receivable, controle de acesso RBAC, pausabilidade, whitelist com restrição de transferência e suporte a metadados URI (IPFS).

1.1 Visão Geral do Escopo

Propriedade	Detalhe
Nome do Projeto	ERC1155ReceivableToken
Cliente	AxiaDigitalSolutions
Linguagem / Ecossistema	Solidity ^0.8.0 / EVM
Arquivos no Escopo	`ERC1155ReceivableToken.sol`
Linhas de Código (SLOC)	189 (código-fonte), 10 funções expostas
Dependências	OpenZeppelin (ERC1155, AccessControl, Pausable, Strings)
Padrões	ERC-1155, ERC-165, Multi-Token Standard

1.2 Resumo das Descobertas

Durante a auditoria, foram identificados **8** problemas de segurança. Após o ciclo de correção, **6 das 8 descobertas (6 de 8) foram resolvidas e verificadas** pela HeimdallSec, permanecendo **2 itens de severidade Informational / Gas pendentes** (correção ainda não aplicada pela equipe), distribuídas da seguinte forma:

Severidade	Quantidade	Status de Resolução
Critical	0	**0 Resolvidos**
High	1	**1 Resolvidos**
Medium	3	**3 Resolvidos**
Low	2	**2 Resolvidos**
Informational / Gas	2	**0 Resolvidos**
Total	8	**6 de 8 resolvidas**

2. Metodologia de Auditoria

Nossa auditoria segue uma metodologia estruturada baseada em padrões de mercado, incluindo as diretrizes do **EEA EthTrust Security Levels** e o **OWASP Smart Contract Security Verification Standard (SCSVS)**.

O processo foi executado nas seguintes fases:

1. **Modelagem de Ameaças:** Mapeamento de atores (DEFAULT_ADMIN_ROLE, MINTER_ROLE, BURNER_ROLE, PAUSER_ROLE, URI_SETTER_ROLE), privilégios e vetores de ataque específicos do multi-token ERC-1155.
2. **Análise Automatizada:** Varredura conceitual com padrões de segurança conhecidos (reentrância, controle de acesso, validação de entrada, CEI).
3. **Revisão Manual:** Leitura linha a linha focada em falhas de lógica de negócios, controle de acesso, gerenciamento de URIs e validade da whitelist.

As vulnerabilidades são classificadas utilizando o sistema da **Immunefi (v2.3)**, baseando-se no impacto potencial (ex: perda de fundos, DoS) e na probabilidade de exploração.

2.1 Análise Geral do Contrato

O contrato **ERC1155ReceivableToken** estende `ERC1155`, `AccessControl` e `Pausable` da OpenZeppelin, fornecendo:

- **Mint individual e em lote:** Apenas `MINTER_ROLE` pode criar tokens, quando não pausado.
- **Burn individual e em lote:** Apenas `BURNER_ROLE` pode queimar tokens, quando não pausado.
- **Pausabilidade:** `PAUSER_ROLE` pode pausar/despausar operações de mint/burn.
- **Whitelist:** `DEFAULT_ADMIN_ROLE` pode gerenciar lista branca de endereços autorizados.
- **Restrição de transferência:** Quando `transferRestricted == true` (padrão), apenas endereços na whitelist podem transferir.
- **Metadados URI:** `URI_SETTER_ROLE` pode definir baseURI e URIs individuais por tokenId (IPFS).

Funções Expostas:

Função	State Mutability	Privilégio	Descrição
`mint(address, uint256, uint256, bytes)`	`nonpayable`	MINTER_ROLE	Cria tokens de um tokenId específico
`mintBatch(address, uint256[], uint256[], bytes)`	`nonpayable`	MINTER_ROLE	Cria múltiplos tokenIds em lote
`burn(address, uint256, uint256)`	`nonpayable`	BURNER_ROLE	Queima tokens de um tokenId
`burnBatch(address, uint256[], uint256[])`	`nonpayable`	BURNER_ROLE	Queima múltiplos em lote
`pause()`	`nonpayable`	PAUSER_ROLE	Pausa operações de mint/burn
`unpause()`	`nonpayable`	PAUSER_ROLE	Despausa operações
`setBaseURI(string)`	`nonpayable`	URI_SETTER_ROLE	Define baseURI para metadados
`setTokenURI(uint256, string)`	`nonpayable`	URI_SETTER_ROLE	Define URI individual por tokenId
`updateWhitelist(address, bool)`	`nonpayable`	DEFAULT_ADMIN_ROLE	Gerencia whitelist
`setTransferRestricted(bool)`	`nonpayable`	DEFAULT_ADMIN_ROLE	Ativa/desativa restrição de transferência
`uri(uint256)`	`view`	Público	Retorna URI de metadados

3. Descobertas Detalhadas (Findings)

[HSEC-ERT-01] Ausência de `whenNotPaused` em Transferências — Pausa Não Bloqueia Transferências ERC-1155

Severidade: High **Status: Resolved Localização:** `ERC1155ReceivableToken.sol`, Função `_beforeTokenTransfer()` (Linhas `162-175`)

Descrição: O modificador `whenNotPaused` é aplicado apenas às funções de mint e burn (`mint()`, `mintBatch()`, `burn()`, `burnBatch()`), mas NÃO é verificado nas transferências comuns (`safeTransferFrom()` e `safeBatchTransferFrom()` herdadas do ERC-1155). O hook `_beforeTokenTransfer()` não possui nenhuma verificação de pausa.

Isso significa que, quando o contrato está pausado:

- **Mint e burn são bloqueados ✓** (correto)
- **Transferências de usuário NÃO são bloqueadas ✗** (risco)

Em cenários de emergência (comprometimento, migração forçada), a intenção de pausar o contrato é congelar todos os movimentos de token — mas as transferências P2P continuam funcionando normalmente para todos os tokenIds.

Cenário de Exploração (Impacto):

1. Equipe detecta atividade maliciosa envolvendo um receivable específico e chama `pause()`.
2. Atacante ainda consegue transferir seus receivables para outra wallet (dificultando recuperação e compliance).
3. Em um cenário multi-token (vários receivables no mesmo contrato), um receivable comprometido pode ser movido mesmo com o contrato pausado.

Recomendação de Mitigação: Adicionar verificação `whenNotPaused` dentro de `_beforeTokenTransfer()`:

```
function _beforeTokenTransfer(
    address operator,
    address from,
    address to,
    uint256[] memory ids,
    uint256[] memory amounts,
    bytes memory data
) internal virtual override {
    super._beforeTokenTransfer(operator, from, to, ids, amounts, data);

    if (from != address(0) && to != address(0)) {
        _requireNotPaused();  // ← NOVA verificação
        if (transferRestricted) {
            require(whitelist[from] || whitelist[to],
                "ERC1155ReceivableToken: transfer restricted to whitelisted addresses");
        }
    }
}
```

Correção Aplicada: A verificação de pausa foi incorporada ao hook `_beforeTokenTransfer()`, chamando `_requireNotPaused()` quando `from != address(0) && to != address(0)`. Com isso, as transferências P2P (`safeTransferFrom()` e `safeBatchTransferFrom()`) passam a ser bloqueadas enquanto o contrato estiver pausado, alinhando o comportamento de pausa a mint e burn.

Verificação da Correção: ✓ Verificado. Confirmado que, com o contrato pausado, as transferências revertem corretamente e voltam a operar após `unpause()`. A pausa agora congela todos os movimentos de token.

Resposta da Equipe (Opcional): Correção implementada conforme recomendado; `_requireNotPaused()` adicionado ao `_beforeTokenTransfer()`.

[HSEC-ERT-02] MintBatch Sem Limite de Tamanho de Array — Possível Block Gas Limit DoS

Severidade: Medium **Status: Resolved Localização:** `ERC1155ReceivableToken.sol`, Função `mintBatch()` (Linhas `68-75`)

Descrição: A função `mintBatch()` aceita arrays `ids[]` e `amounts[]` sem qualquer limite de tamanho. Se um array muito grande for passado, a transação pode exceder o block gas limit (30M na maioria das EVM chains), resultando em:

- Falha da transação após consumir todo o gas.
- Impossibilidade de mintar um lote grande em uma única transação.
- Necessidade de o operador dividir manualmente em múltiplas transações.

Embora não seja uma vulnerabilidade de segurança no sentido estrito, este é um problema de resiliência operacional que pode causar frustração e custos desnecessários de gas para operadores.

Cenário de Exploração (Impacto):

1. Operador prepara mintBatch com 200+ receivables de uma só vez.
2. Transação falha após consumir 5M+ gas (parcialmente desperdiçado).
3. O operador não sabe qual foi o ponto de falha — precisa tentar com lotes menores.
4. Em chains com block gas limit baixo (ex: Polygon: 30M), o problema é agravado.

Recomendação de Mitigação: Adicionar verificação de tamanho máximo de arrays:

```
uint256 public constant MAX_BATCH_SIZE = 50;

function mintBatch(
    address to,
    uint256[] memory ids,
    uint256[] memory amounts,
    bytes memory data
) external onlyRole(MINTER_ROLE) whenNotPaused {
    require(ids.length <= MAX_BATCH_SIZE, "ERC1155ReceivableToken: batch too large");
    require(ids.length == amounts.length, "ERC1155: ids and amounts length mismatch");
    _mintBatch(to, ids, amounts, data);
}
```

Correção Aplicada: Foi adicionada a constante `MAX_BATCH_SIZE = 50` e a verificação `require(ids.length <= MAX_BATCH_SIZE, ...)` em `mintBatch()` (e, de forma equivalente, em `burnBatch()`), impedindo lotes grandes o suficiente para exceder o block gas limit. A validação de igualdade entre `ids.length` e `amounts.length` foi mantida.

Verificação da Correção: ✓ Verificado. Lotes acima de 50 elementos revertem com a mensagem "batch too large" antes de consumir gas significativo, eliminando o risco de DoS por block gas limit.

Resposta da Equipe (Opcional): Limite de lote de 50 aplicado a `mintBatch()` e `burnBatch()`.

[HSEC-ERT-03] Risco de Centralização — Admin Único com Controle Total Sobre TokenIds e URIs

Severidade: Medium **Status: Resolved Localização:** `ERC1155ReceivableToken.sol`, Constructor (Linhas `36-47`) e Funções de URI (Linhas `119-130`)

Descrição: O construtor concede `DEFAULT_ADMIN_ROLE`, `MINTER_ROLE`, `BURNER_ROLE`, `PAUSER_ROLE` e `URI_SETTER_ROLE` a um único endereço `admin`. Isso significa que uma única chave privada tem controle total sobre:

- Criar tokens ilimitados de qualquer receivable (MINTER_ROLE).
- Queimar tokens de qualquer receivable (BURNER_ROLE).
- Pausar/despausar o contrato (PAUSER_ROLE).
- Gerenciar whitelist (DEFAULT_ADMIN_ROLE).
- **Alterar URIs de metadados a qualquer momento (URI_SETTER_ROLE)** — inclusive após os tokens terem sido distribuídos.

O poder de alterar URIs após a distribuição é particularmente crítico para receivables, onde a URI pode representar contratos de crédito, documentos de cessão ou comprovantes de pagamento. Alterar a URI de um receivable já emitido pode:

- Fazer com que um receivable aponte para metadados fraudulentos.
- Quebrar a integridade de referências off-chain (sistemas legados, APIs de crédito).
- Prejudicar a confiança dos investidores no sistema.

Cenário de Exploração (Impacto):

1. Chave privada do admin é comprometida.
2. Atacante chama `setTokenURI(42, "ipfs://malicious-json")` para um receivable já distribuído.
3. Investidores consultam `uri(42)` e recebem metadados fraudulentos.
4. Sistemas off-chain que dependem da URI para validação de crédito são enganados.

Recomendação de Mitigação:

1. **Separar papéis:** `URI_SETTER_ROLE` em endereço diferente de `MINTER_ROLE` e `BURNER_ROLE`.
2. **Multisig + Timelock para setTokenURI/setBaseURI:**

```
mapping(uint256 => uint256) public pendingURITimelock;

function scheduleSetTokenURI(uint256 tokenId, string memory tokenURI)
    external onlyRole(URI_SETTER_ROLE)
{
    pendingURITimelock[tokenId] = block.timestamp + 2 days;
    emit TokenURIChangeQueued(tokenId, tokenURI, pendingURITimelock[tokenId]);
}

function executeSetTokenURI(uint256 tokenId, string memory tokenURI)
    external onlyRole(URI_SETTER_ROLE)
{
    require(block.timestamp >= pendingURITimelock[tokenId], "Timelock not expired");
    _tokenURIs[tokenId] = tokenURI;
    emit TokenURIUpdated(tokenId, tokenURI);
}
```

1. **Considerar tornar URIs imutáveis após o primeiro mint** (a menos que o modelo de negócios exija updates):

```
function mint(address to, uint256 id, uint256 amount, bytes memory data)
    external onlyRole(MINTER_ROLE) whenNotPaused
{
    require(bytes(_tokenURIs[id]).length > 0, "ERC1155ReceivableToken: URI must be set before
mint");
    _mint(to, id, amount, data);
}
```

Correção Aplicada: O `URI_SETTER_ROLE` foi separado dos papéis de mint/burn e as alterações de metadados (`setTokenURI`/`setBaseURI`) passaram a operar sob um mecanismo de timelock de 2 dias (agendamento + execução), com emissão dos eventos `TokenURIChangeQueued` e `TokenURIUpdated`. Adicionalmente, o mint passou a exigir URI previamente definida, reduzindo a janela para substituição arbitrária de metadados de receivables já distribuídos.

Verificação da Correção: ✓ Verificado. Confirmado que alterações de URI só se efetivam após expirar o timelock e que os papéis foram atribuídos a endereços distintos (multisig para administração). O risco de alteração instantânea de metadados por chave única foi mitigado.

Resposta da Equipe (Opcional): Papéis separados e timelock de 2 dias implementado para alterações de URI.

[HSEC-ERT-04] Risco de Centralização — Admin Único com Todos os Papéis (Single Point of Failure)

Severidade: Medium **Status: Resolved Localização:** `ERC1155ReceivableToken.sol`, Constructor (Linhas `36-47`)

Descrição: Além dos riscos específicos de URI (HSEC-ERT-03), a centralização de todos os 5 papéis em um único endereço amplifica o impacto de um comprometimento:

- **DEFAULT_ADMIN_ROLE:** Gerencia whitelist, restrição de transferência e concede/revoga papéis.
- **MINTER_ROLE:** Cria tokens de qualquer receivable (inflação ilimitada).
- **BURNER_ROLE:** Queima tokens de qualquer receivable de qualquer holder.
- **PAUSER_ROLE:** Pausa todas as operações de mint/burn.
- **URI_SETTER_ROLE:** Altera metadados de qualquer receivable.

Um atacante com a chave do admin pode:

1. Fazer mint de receivables inexistentes.
2. Alterar URIs para ocultar fraudes.
3. Desativar whitelist e transferir tudo.
4. Queimar tokens de investidores específicos.
5. Pausar mint/burn para impedir novos receivables legítimos.

Recomendação de Mitigação:

1. Distribuir papéis entre diferentes EOAs e/ou multisigs.
2. Implementar hierarquia de papéis: `DEFAULT_ADMIN_ROLE` em multisig, demais papéis delegados a operadores específicos.
3. Considerar rate limiting para `MINTER_ROLE` (limite de tokens mintados por período).

```
mapping(address => uint256) public lastMintTimestamp;
mapping(address => uint256) public mintedInWindow;
uint256 public constant MINT_WINDOW = 1 days;
uint256 public constant MINT_LIMIT = 1000000 * 10**18; // 1M tokens

function mint(address to, uint256 id, uint256 amount, bytes memory data)
    external onlyRole(MINTER_ROLE) whenNotPaused
{
    _checkMintRateLimit(amount);
    _mint(to, id, amount, data);
}

function _checkMintRateLimit(uint256 amount) internal {
    if (block.timestamp > lastMintTimestamp[_msgSender()] + MINT_WINDOW) {
        mintedInWindow[_msgSender()] = 0;
        lastMintTimestamp[_msgSender()] = block.timestamp;
    }
    mintedInWindow[_msgSender()] += amount;
    require(mintedInWindow[_msgSender()] <= MINT_LIMIT, "Mint limit exceeded");
}
```

Correção Aplicada: Os cinco papéis foram distribuídos entre endereços distintos: `DEFAULT_ADMIN_ROLE` passou a ser controlado por um multisig (Safe), com os demais papéis delegados a operadores específicos, eliminando o single point of failure. Foi implementado rate limiting para `MINTER_ROLE` (`MINT_WINDOW`/`MINT_LIMIT`), limitando a quantidade de tokens mintados por janela de tempo.

Verificação da Correção: ✓ Verificado. Confirmada a hierarquia de papéis com `DEFAULT_ADMIN_ROLE` em multisig e o rate limiting revertendo mints acima do limite por janela. O impacto de um eventual comprometimento de chave foi significativamente reduzido.

Resposta da Equipe (Opcional): Papéis distribuídos entre multisig e operadores; rate limiting de mint implementado.

[HSEC-ERT-05] URI Configurável Após Mint — Risco de Integridade de Metadados

Severidade: Low **Status: Resolved Localização:** `ERC1155ReceivableToken.sol`, Funções `setBaseURI()` e `setTokenURI()` (Linhas `119-130`)

Descrição: A função `uri(uint256 tokenId)` (Linhas `135-141`) primeiro verifica se existe uma URI individual (`_tokenURIs[tokenId]`). Se existir, retorna essa URI. Caso contrário, constrói a URI a partir da `baseURI` concatenada com o `tokenId`.

O problema é que `setBaseURI()` pode ser chamada a qualquer momento por `URI_SETTER_ROLE`. Isso significa que a URI de UM token pode mudar entre duas consultas, mesmo que `setTokenURI()` nunca tenha sido chamada para aquele tokenId específico.

Para tokens que dependem de `baseURI` (sem `_tokenURIs` individual), a integridade do metadado é totalmente dependente da boa-fé do `URI_SETTER_ROLE`.

Além disso, `_tokenURIs[tokenId]` nunca pode ser resetada — uma vez definida, a URI individual sempre terá precedência sobre `baseURI`. Isso é um comportamento correto, mas significa que erros em `setTokenURI()` são irreversíveis (a menos que chamem novamente com outra URI).

Cenário de Exploração (Impacto):

1. Receivables são mintados com URIs baseadas em `baseURI` (ex: `ipfs://QmBase/1`, `ipfs://QmBase/2`).
2. `URI_SETTER_ROLE` altera `baseURI` para um novo endpoint malicioso.
3. Todos os tokens sem URI individual agora apontam para metadados diferentes.
4. Investidores que verificam a URI após meses podem receber dados incorretos.

Recomendação de Mitigação:

1. **Fixar baseURI no constructor e remover `setBaseURI()`:**

```
string private _baseURI;  // Imutável após constructor

constructor(string memory baseURI_, address admin) ERC1155(baseURI_) {
    _baseURI = baseURI_;
    // ... setup de roles
}

// REMOVER setBaseURI() do código
```

1. **Ou adicionar timelock para alterações de baseURI** (como recomendado no HSEC-ERT-03).

Correção Aplicada: As alterações de `baseURI` passaram a estar sujeitas ao mesmo mecanismo de timelock adotado no HSEC-ERT-03, impedindo trocas instantâneas do endpoint de metadados. Para tokens que dependem de `baseURI`, a mudança agora exige agendamento e período de espera antes de efetivar-se, preservando a integridade dos metadados de receivables já distribuídos.

Verificação da Correção: ✓ Verificado. Confirmado que `setBaseURI()` só produz efeito após o timelock e que a precedência de `_tokenURIs` individuais foi mantida. A integridade de metadados baseados em `baseURI` deixou de depender exclusivamente da boa-fé do `URI_SETTER_ROLE`.

Resposta da Equipe (Opcional): Timelock aplicado às alterações de `baseURI`, alinhado ao HSEC-ERT-03.

[HSEC-ERT-06] Mint de TokenId Sem URI Definida — Token "Fantasma" Sem Metadados

Severidade: Low **Status: Resolved Localização:** `ERC1155ReceivableToken.sol`, Função `mint()` (Linhas `56-63`)

Descrição: A função `mint()` não verifica se o `tokenId` possui metadados definidos (seja via `_tokenURIs` ou `baseURI`). É possível mintar um token com um `tokenId` arbitrário que não possui URI associada.

- Se `_baseURI` for uma string vazia (caso padrão se não configurada corretamente), `uri(tokenId)` retornará uma string vazia ou apenas o número do tokenId.

- Sistemas off-chain (exploradores, marketplaces, APIs de crédito) podem não conseguir exibir o receivable corretamente.
- Tokens mintados sem URI podem causar confusão em auditorias — não é possível distinguir entre receivable sem documento e receivable com documento perdido.

Cenário de Exploração (Impacto):

1. Operador com `MINTER_ROLE` acidentalmente min tou sem antes configurar a URI.
2. Token mintado é transferido para investidor.
3. Investidor não consegue ver os detalhes do receivable.
4. Para corrigir, URI_SETTER_ROLE precisa chamar `setTokenURI()` — mas o dano de confiança já ocorreu.

Recomendação de Mitigação: Validar que a URI está configurada antes do mint:

```
function mint(address to, uint256 id, uint256 amount, bytes memory data)
    external onlyRole(MINTER_ROLE) whenNotPaused
{
    require(bytes(uri(id)).length > 0, "ERC1155ReceivableToken: URI not set");
    _mint(to, id, amount, data);
}
```

Alternativa: Exigir que `setTokenURI()` seja chamada antes do mint, tornando a URI obrigatória para cada tokenId.

Correção Aplicada: A função `mint()` passou a validar que a URI está configurada antes de criar o token, via `require(bytes(uri(id)).length > 0, "ERC1155ReceivableToken: URI not set")`. Assim, tornou-se impossível mintar um receivable "fantasma" sem metadados, garantindo que todo tokenId emitido possua URI resolvível.

Verificação da Correção: ✓ Verificado. Confirmado que mints de tokenIds sem URI configurada revertem, enquanto tokenIds com URI definida (individual ou via `baseURI`) são mintados normalmente.

Resposta da Equipe (Opcional): Validação de URI obrigatória adicionada ao `mint()`.

[HSEC-ERT-07] Ausência de Função `renounceRole` — Possibilidade de Auto-Revogação Irreversível

Severidade: Informational **Status: Unresolved Localização:** `ERC1155ReceivableToken.sol`, Herda `AccessControl` da OpenZeppelin

Descrição: O contrato herda `AccessControl` da OpenZeppelin, que inclui a função `renounceRole(bytes32 role, address account)`. Esta função permite que um endereço abra mão de seu próprio papel de forma irreversível.

Se o `admin` (DEFAULT_ADMIN_ROLE) acidentalmente chamar `renounceRole(DEFAULT_ADMIN_ROLE, adminAddress)`:

- Ninguém mais pode gerenciar whitelist.
- Ninguém mais pode gerenciar URIs.
- Ninguém mais pode adicionar novos minters/burners/pausers.
- O contrato se torna permanentemente imutável ou inutilizável.

O impacto em um contrato ERC-1155 com múltiplos receivables é particularmente severo, pois sem `URI_SETTER_ROLE` ninguém pode configurar URIs de novos receivables, congelando a operação.

Recomendação de Mitigação: Sobrescrever `renounceRole` para bloquear renúncia de `DEFAULT_ADMIN_ROLE`:

```
function renounceRole(bytes32 role, address account)
    public virtual override
{
    if (role == DEFAULT_ADMIN_ROLE) {
        revert("ERC1155ReceivableToken: cannot renounce DEFAULT_ADMIN_ROLE");
```

```
    }
    if (role == URI_SETTER_ROLE) {
        revert("ERC1155ReceivableToken: cannot renounce URI_SETTER_ROLE");
    }
    super.renounceRole(role, account);
}
```

Resposta da Equipe: Pendente — correção ainda não aplicada pela equipe.

[HSEC-ERT-08] Versão do Solidity com Pragmatismo Amplo — ^0.8.0 Permite Versões Instáveis

Severidade: Informational **Status: Unresolved Localização:** `ERC1155ReceivableToken.sol`, Linha `2`

Descrição: O contrato utiliza `pragma solidity ^0.8.0`, que permite qualquer versão 0.8.x. O OpenZeppelin v5.x (ERC1155, AccessControl) tem requisitos mínimos de versão mais altos, e compilar com versões 0.8.0-0.8.4 pode causar:

- Erros de compilação com OpenZeppelin moderno.
- Diferenças de comportamento em ABI encoding.
- Inconsistência entre ambientes de teste e produção.

Recomendação de Mitigação: Fixar a versão para uma específica e testada:

```
pragma solidity ^0.8.24;
```

Resposta da Equipe: Pendente — correção ainda não aplicada pela equipe.

4. Recomendações Gerais e Melhores Práticas

Além das vulnerabilidades específicas listadas acima, a equipe da HeimdallSec identificou áreas onde a postura de segurança do protocolo pode ser melhorada a longo prazo:

1. **Governança Multi-Assinatura:** Todos os papéis administrativos devem ser geridos por contratos Multisig (Safe). Sob nenhuma circunstância uma EOA única deve possuir `DEFAULT_ADMIN_ROLE` ou `URI_SETTER_ROLE` em produção.
2. **Imutabilidade de Metadados:** Para receivables, considere tornar as URIs imutáveis após o primeiro mint. A integridade dos documentos de crédito é fundamental para a confiança dos investidores.
3. **Separação de Papéis Operacionais:** Implementar uma separação clara:

- Equipe de Operações: `MINTER_ROLE` (criar novos receivables).
- Equipe de Compliance: `PAUSER_ROLE` (emergência).
- Equipe de Administração: `DEFAULT_ADMIN_ROLE` + `URI_SETTER_ROLE` (multisig).
- Sem `BURNER_ROLE` ativo (apenas em emergências, via multisig).

1. **Pausa Deve Congelar Tudo:** Corrigir `_beforeTokenTransfer()` para incluir `_requireNotPaused()`. Sem essa correção, o mecanismo de pausa é ineficaz.
2. **Monitoramento de URI Changes:** Implementar monitoramento off-chain para eventos `TokenURIUpdated`, notificando a equipe de segurança imediatamente sobre alterações suspeitas de metadados.
3. **Testes de Integridade de Metadados:** Criar testes que validam:

- URI retorna valor correto antes e depois do mint.
- Base URI e token URI individuais funcionam conforme esperado.
- Alterações de URI após mint são detectáveis via eventos.

1. **Limites de Batch:** Definir tamanho máximo para `mintBatch()` e `burnBatch()` para evitar block gas limit issues.
2. **Dependências:** Manter OpenZeppelin atualizado para a versão mais recente.

5. Disclaimer — Análise de Ameaças por Ação

Abaixo, um resumo do modelo de ameaças por ação privilegiada:

Ação	Quem Pode Fazer	Impacto Potencial	Mitigação Recomendada
Mint / MintBatch	MINTER_ROLE	Criação fraudulenta de receivables	Rate limiting + multisig
Burn / BurnBatch	BURNER_ROLE	Destruição unilateral de receivables	Multisig only
Pause / Unpause	PAUSER_ROLE	DoS temporário	Multisig + extender para transfers
UpdateWhitelist	DEFAULT_ADMIN_ROLE	Exclusão de investidores	Timelock + multisig
SetTransferRestricted	DEFAULT_ADMIN_ROLE	Desligamento total de proteções	Timelock + multisig
SetBaseURI	URI_SETTER_ROLE	Alteração de metadados em massa	Timelock + imutabilidade pós-mint
SetTokenURI	URI_SETTER_ROLE	Alteração de metadados individual	Timelock
RenounceRole	Qualquer role holder	Perda irreversível de funcionalidades	Bloquear renúncia de papéis críticos

6. Termo de Isenção de Responsabilidade (Disclaimer)

Este relatório de auditoria não constitui um endosso, garantia ou certificação absoluta de que o código auditado está livre de vulnerabilidades ou falhas. A auditoria representa um esforço de tempo limitado (time-boxed) para identificar o máximo possível de problemas de segurança utilizando o conhecimento e as ferramentas disponíveis no momento da execução.

A HeimdallSec não se responsabiliza por perdas financeiras, danos operacionais ou explorações futuras que possam ocorrer no protocolo.

gov.br
Documento assinado digitalmente
KLEBER AQUINO DOS SANTOS JUNIOR
Data: 24/07/2026 21:18:39-0300
Verifique em https://validar.iti.gov.br